Lecture 24: Multicasting

Objectives:

- Learn what Multicasting is about
- Learn about the IP Addresses used for Multicasting
- Learn about the Internet Group Management Protocol (IGMP) used for multicasting
- Learn how C# supports multicasting

1. What is Multicasting?

Broadcasting is an excellent way to send information to all devices on a subnet. However, it has one serious drawback:

- The broadcast packets are restricted to the local subnet.

Multicasting was designed to address this drawback.

Multicasting allows application programs to send a single packet to a select subset of devices called a **multicast group**.

Each multicast group is identified by a single special IP address discussed below.

A packet sent with the particular IP address as the destination address will be received by each member of the group.

Multicast group is a dynamic group. That is, members can join and leave the group at any time. Also members of a multicast group can span across network boundaries.

Examples of application areas that can take advantage of multicasting include videoconferencing, e-learning, stock quotes, news, etc.

2. Multicast IP Addresses

IP multicasting uses a particular range of IP addresses to designate different multicast groups.

The class D IP addresses in the range: 224.0.0.0 through 239.255.255.255 are used to represent multicast groups.

However, some of these addresses are reserved for special purposes as discussed below:

Local Control Block:

Addresses in the range: 224.0.0.0 through 224.0.0.255 are reserved for used by network protocols on a local network.

For example, 224.0.0.1 represents all systems on this subnet. 224.0.0.2 represents all routers on this subnet.

Global Scope:

Addresses in the range: 224.0.1.0 through 238.255.255.255 are called globally scoped addresses. That is, they can be used to multicast data across the Internet.

Limited Scope:

Addresses in the range 239.0.0.0 through 239.255.255.255 are called limited scope addresses.

Routers are normally configured to prevent multicast traffic with these addresses from crossing over the local network.

More detailed information about multicast addresses can be found at: http://www.iana.org/assignments/multicast-addresses

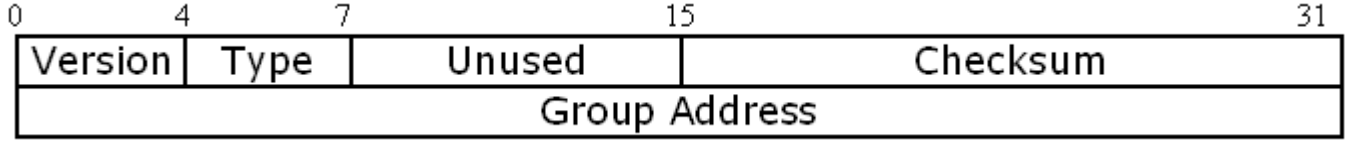
3. Internet Group Management Protocol (IGMP)

IGMP is used to dynamically register individual hosts in a multicast group on a particular LAN.

There are two versions of the protocol as discussed below:

IGMP Version 1 (RFC 1112)

The following figure shows the format of IGMP version 1 packet.



In Version 1, there are just two different types of IGMP messages:

- Membership query
- Membership report

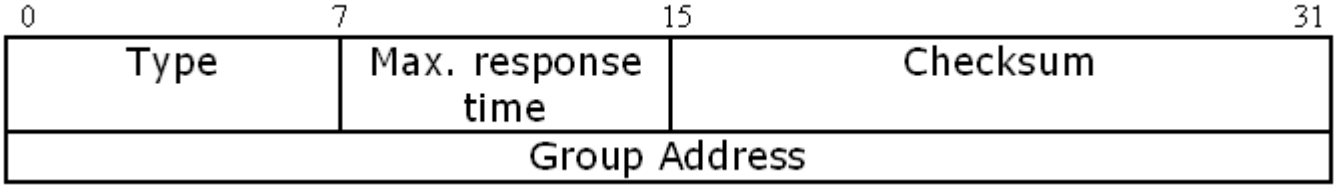
Hosts send IGMP membership report message corresponding to a particular multicast group to indicate that they are interested in joining that group.

Membership query message is sent by routers periodically to verify that at least one host on the subnet is still interested in receiving traffic directed to a particular multicast group.

When there is no reply to three consecutive IGMP membership queries, the router times-out the group and stops forwarding traffic directed towards that group.

IGMP Version 2 (RFC 2236)

The following figure shows the format of IGMP version 2 packet.



In Version 2, there are four types of IGMP messages:

- Membership query
- Version 1 membership report
- Version 2 membership report
- Leave group

IGMP Version 2 works basically the same as Version 1. The main difference is that there is a leave group message.

The hosts now can communicate to the local multicast router to indicate their intention to leave the group.

The router then sends a group-specific query to determine whether there are any remaining hosts interested in receiving the traffic.

If there are no replies, the router times-out the group and stops forwarding the traffic. This can greatly reduce the leave latency compared to IGMP Version 1.

4. C# Support for Multicast

The .NET network library supports IP multicasting by using Socket options.

There are two socket options that are used to join and to leave a multicast group respectively.

These options are defined by IP-level Socket options names, AddMembership and DropMembership.

The value for each of these options is an instance of the MulticastOption class. This class has two constructors as follows:

`MulticastOption(IPAddress)`	IPAddress specifies the multicast group address. If the machine has more than one interface, all of them are affected by the socket option.
`MulticastOption(IPAddress, IPAddress)`	The second address specifies the interface to be affected by the socket option.

The following example shows a simple multicast receiver application:

```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;

public class SimpleMulticastReceiver
{
   public static void Main()
   {
      Socket sock = new Socket(AddressFamily.InterNetwork,
SocketType.Dgram, ProtocolType.Udp);
      Console.WriteLine("Ready to receive...");

      IPEndPoint localEP = new IPEndPoint(IPAddress.Any, 9090);
      sock.Bind(localEP);
      sock.SetSocketOption(SocketOptionLevel.IP,
SocketOptionName.AddMembership,
         new MulticastOption(IPAddress.Parse("224.100.0.1")));
```

```
      EndPoint remoteEP = new IPEndPoint(IPAddress.Any, 0);
      byte[] data;
      string message;
      while (true) {
         data = new byte[1024];
         int recv = sock.ReceiveFrom(data, ref remoteEP);
         message = Encoding.ASCII.GetString(data, 0, recv);
         Console.WriteLine("received: {0}  from: {1}", message,
remoteEP.ToString());
      }
   }
}
```

Notes: The following ere important points to note about multicast sockets:

- The SetSocketOption method must be called after the call to Bind method. This enables the multicast group to be set for a specific IPEndPoint.
- Once the socket has been added to a specific multicast group, the ReceiveFrom method will accept packets destined for each of the following:
  - The IPEndPoint specified in the call to the Bind method.
  - The multicast group IP address specified in the MulticastOption constructor
  - Broadcast packets for the specified IPEndPoint.

Thus, applications are not guaranteed to receive packets destined just for the multicast group and there is no easy way of distinguishing these packets.

Sending Multicast Packets to local subnet:

Nothing special must be done to send packets to members of a multicast group in the current subnet; you just specify the multicast group IP address as the destination address as the following example shows.

```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;

public class SimpleMulticastSender {
   public static void Main()     {
      Socket sock = new Socket(AddressFamily.InterNetwork,
SocketType.Dgram, ProtocolType.Udp);
```

```
      IPEndPoint endPoint = new
IPEndPoint(IPAddress.Parse("224.100.0.1"), 9090);

      String message;
      while (true) {
         Console.Write("Enter message to Multicast: ");
         message = Console.ReadLine();

         if (message=="")
            break;

         byte[] data = Encoding.ASCII.GetBytes(message);
         sock.SendTo(data, endPoint);
      }
      sock.Close();
   }
}
```

Sending Multicast Packets outside local subnet:

By default, multicast packets sent by the socket class have a TTL value of one, meaning they cannot be forwarded by the router to another network.

To send multicast packets that can traverse multiple routers, the socket has to:

- Join the multicast group
- Increase the TTL value.

Increasing the TTL value is again done using the SetSocketOption() method.

The required socket option is IP-level and its name is MulticastTimeToLive. The value is any positive integer value.

As with receiving multicast sockets, the socket must be bound to a local end point before setting the socket option.

The following example modifies the SimpleMulticast sender to allow for sending packets across networks.

```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;
```

```
public class BetterMulticastSender {
   public static void Main()     {
      Socket sock = new Socket(AddressFamily.InterNetwork,
SocketType.Dgram, ProtocolType.Udp);
      IPEndPoint localEP = new IPEndPoint(IPAddress.Any, 0);
      sock.Bind(localEP);
      sock.SetSocketOption(SocketOptionLevel.IP,
SocketOptionName.AddMembership,
         new MulticastOption(IPAddress.Parse("224.100.0.1")));
      sock.SetSocketOption(SocketOptionLevel.IP,
SocketOptionName.MulticastTimeToLive, 50);


      IPEndPoint multicastEP = new
IPEndPoint(IPAddress.Parse("224.100.0.1"), 9090);
         String message;
      while (true) {
         Console.Write("Enter message to Multicast: ");
         message = Console.ReadLine();

         if (message=="")
            break;

         byte[] data = Encoding.ASCII.GetBytes(message);
         sock.SendTo(data, multicastEP);
      }
      sock.Close();
   }
}
```

Multicasting using the UdpClient class:

The UdpClient class supports multicasting by providing the following methods:

`JoinMulticastGroup(IPAddress mip)`	Join a multicast group with mip as the group address and default TLL value of 1
`JoinMulticastGroup(IPAddress mip, int ttl)`	Same as above but allows you to provide a TTL value.
`DropMulticastGroup(IPAddress)`	Removes the socket from the multicast group with IPAddress

The following examples show how to use the UdpClient class to create both a multicast receiver and sender.

```
using System;
using System.Net;
```

```
using System.Net.Sockets;
using System.Text;

class UdpClientMulticastReceiver {
  public static void Main()   {
     UdpClient sock = new UdpClient(9050);
     Console.WriteLine("Ready to receive...");

     sock.JoinMulticastGroup(IPAddress.Parse("224.100.0.1"), 50);

     IPEndPoint remoteEP = new IPEndPoint(IPAddress.Any, 0);

     byte[] data;
     string message;
     while (true) {
        data = sock.Receive(ref remoteEP);
        message = Encoding.ASCII.GetString(data, 0, data.Length);
        Console.WriteLine("received: {0}  from: {1}", message,
remoteEP.ToString());
     }
  }
}
```

```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;

class UdpClientMulticastSender {
  public static void Main()   {
     UdpClient sock = new UdpClient(9090);
     IPEndPoint remoteEP = new
IPEndPoint(IPAddress.Parse("224.100.0.1"), 9090);

     byte[] data;
     string message;
     while (true) {
        Console.Write("Enter Message to Multicast: ");
        message = Console.ReadLine();

        if (message == "")
          break;

        data = Encoding.ASCII.GetBytes(message);
        sock.Send(data, data.Length, remoteEP);
     }
     sock.Close();
  }
```

```
}
```

Note that the UdpClientMulticastSender above can only send to a group within the local subnet. To send to a group outside, the socket has to effectively join the group and increase the TTL value.